Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

a) **Write a C code to create a process using fork() system call.**

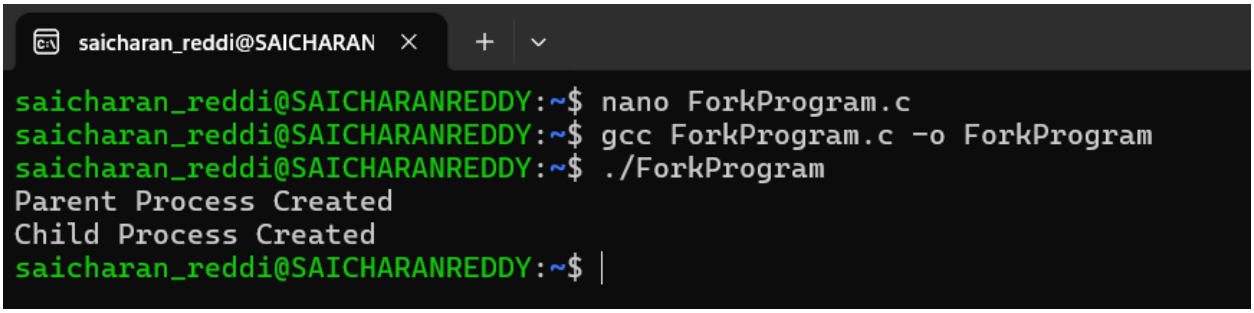
b) **Read the user input as shell command.**

c) **Use a system call exec() to execute the shell command.**

d) **Use a wait() system call for waiting a parent process.**

e) **Display the PIDs of parent and child processes.**

OUTPUT's:



saicharan_reddi@SAICHARAN

GNU nano 8.7.1 ForkProgram.c

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child Process Created\n");
    }
    else if (pid > 0)
    {
        printf("Parent Process Created\n");
    }
    else
    {
        printf("Fork Failed\n");
    }

    return 0;
}
```

saicharan_reddi@SAICHARAN

GNU nano 8.7.1 ForkProgram.c

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child Process Created\n");
    }
    else if (pid > 0)
    {
        printf("Parent Process Created\n");
    }
    else
    {
        printf("Fork Failed\n");
    }

    return 0;
}
```

```
saicharan_reddi@SAICHARANREDDY:~$ nano shellcommand.c
saicharan_reddi@SAICHARANREDDY:~$ gcc shellcommand.c -o shellcommand
saicharan_reddi@SAICHARANREDDY:~$ ./shellcommand
Enter Linux Command: ps
You entered: ps
saicharan_reddi@SAICHARANREDDY:~$
```

saicharan_reddi@SAICHARAN

```
 GNU nano 8.7.1                                          shellcommand.c
#include <stdio.h>

int main()
{
    char command[100];

    printf("Enter Linux Command: ");
    scanf("%99s", command);

    printf("You entered: %s\n", command);

    return 0;
}
```

```
saicharan_reddi@SAICHARANREDDY:~$ nano SystemCall.c
saicharan_reddi@SAICHARANREDDY:~$ gcc SystemCall.c -o SystemCall
saicharan_reddi@SAICHARANREDDY:~$ ./SystemCall
Enter Linux Command: pwd
Executing command...
/home/saicharan_reddi
saicharan_reddi@SAICHARANREDDY:~$
```

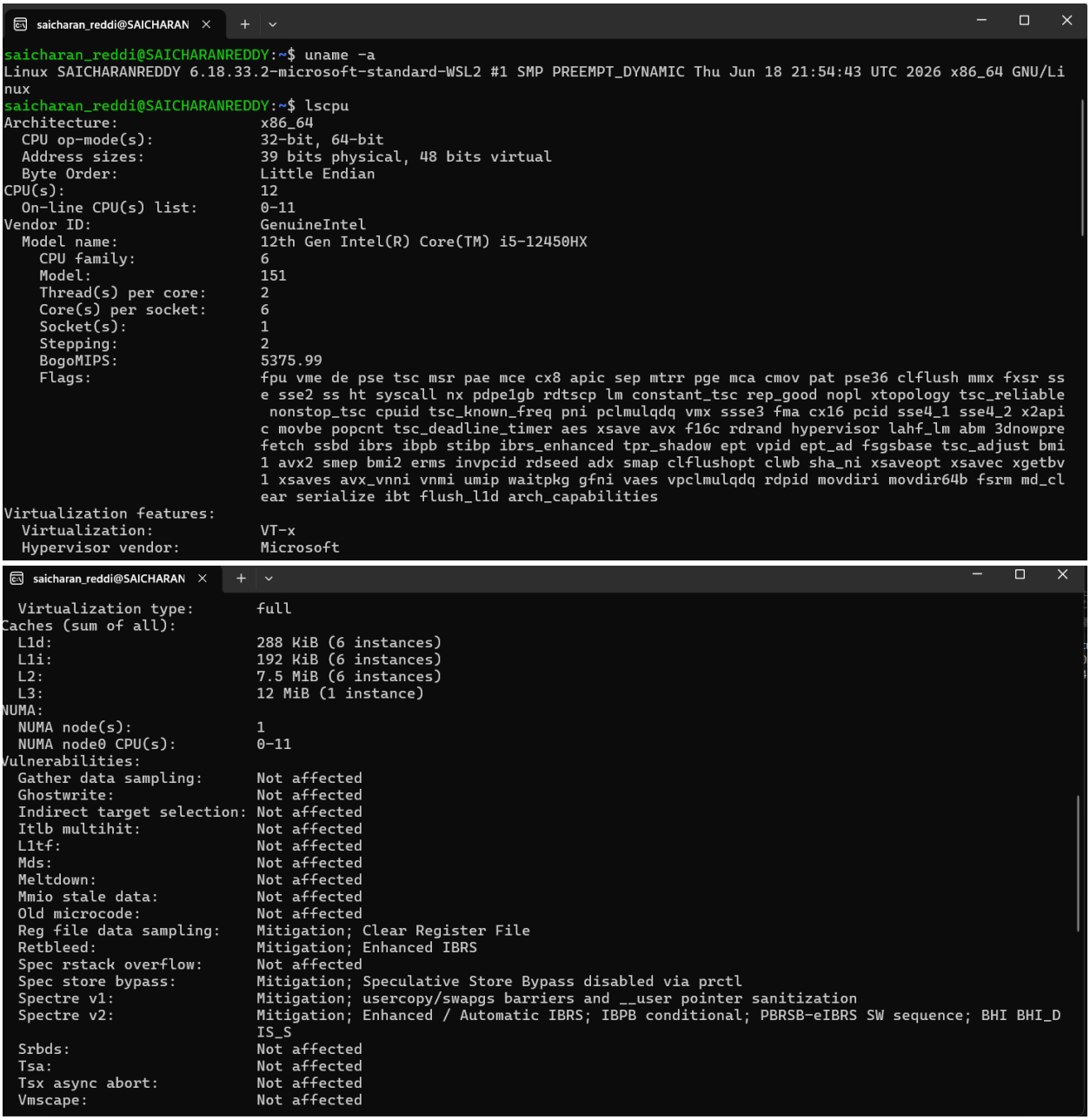
```
  GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    char command[100];
    printf("Enter Linux Command: ");
    scanf("%99s", command);
    printf("Executing command...\n");
    execlp(command, command, NULL);
    perror("Execution Failed");
    return 0;
}
```

```
saicharan_reddi@SAICHARANREDDY:~$ nano SystemCallParent.c
saicharan_reddi@SAICHARANREDDY:~$ gcc SystemCallParent.c -o SystemCallParent
saicharan_reddi@SAICHARANREDDY:~$ ./SystemCallParent
Parent Process is Waiting...
Child Process is Running
Child Process Finished
saicharan_reddi@SAICHARANREDDY:~$
```

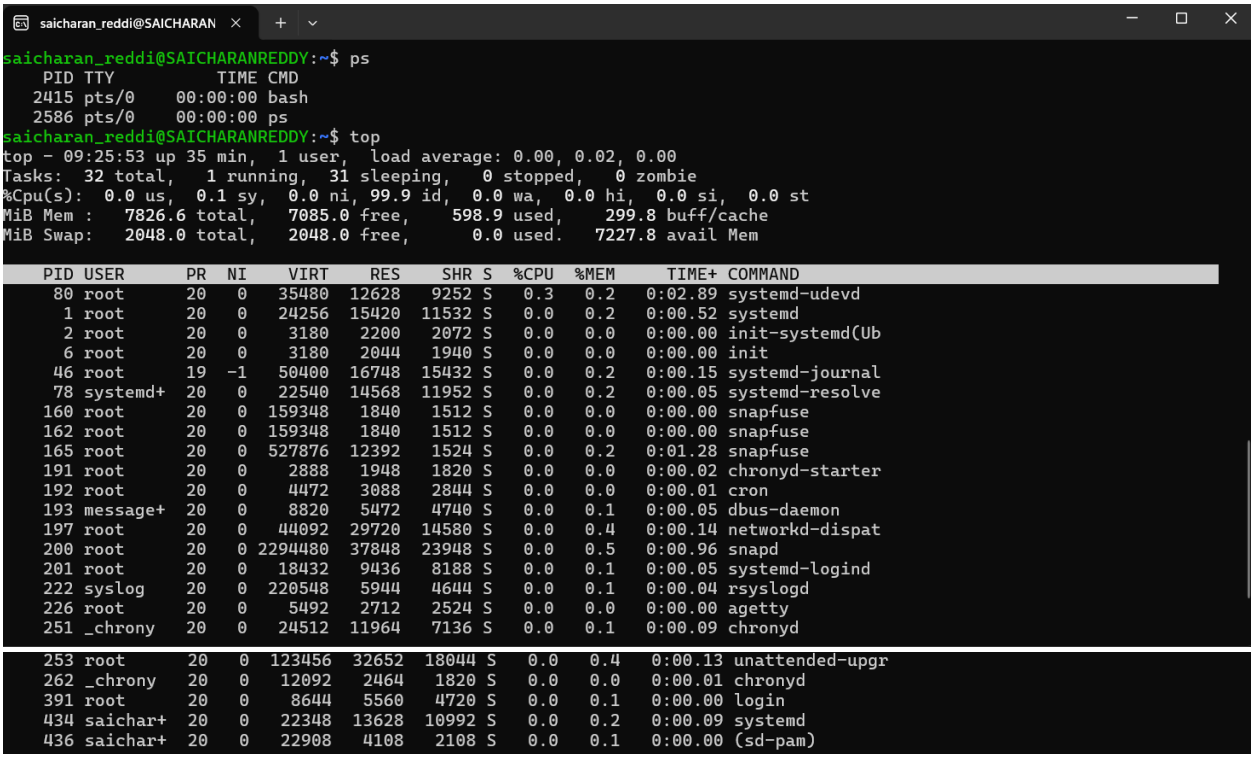
```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child Process is Running\n");
    }
    else if (pid > 0)
    {
        printf("Parent Process is Waiting...\n");

        wait(NULL);

        printf("Child Process Finished\n");
    }
    else
    {
        printf("Fork Failed\n");
    }
```

```
saicharan_reddi@SAICHARAN
saicharan_reddi@SAICHARANREDDY:~$ nano DisplayPID.c
saicharan_reddi@SAICHARANREDDY:~$ gcc DisplayPID.c -oDisplayPID
saicharan_reddi@SAICHARANREDDY:~$ ./DisplayPID
Parent Process
Parent PID: 2522
Child PID: 2523
Child Process
Child PID: 2523
Parent PID: 2413
saicharan_reddi@SAICHARANREDDY:~$
```

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    pid_t pid;

pid = fork();

if (pid == 0)
{
    printf("Child Process\n");
    printf("Child PID : %d\n", getpid());
    printf("Parent PID: %d\n", getppid());
}
else if (pid > 0)
{
    printf("Parent Process\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID  : %d\n", pid);
}
else
{
    printf("Fork Failed\n");
}

return 0;
}
```


saicharan_reddi@SAICHARAN
saicharan_reddi@SAICHARANREDDY:~$ uname -a
Linux SAICHARANREDDY 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
saicharan_reddi@SAICHARANREDDY:~$ lscpu
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 39 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 12
On-line CPU(s) list: 0-11
Vendor ID: GenuineIntel
Model name: 12th Gen Intel(R) Core(TM) i5-12450HX
CPU family: 6
Model: 151
Thread(s) per core: 2
Core(s) per socket: 6
Socket(s): 1
Stepping: 2
BogoMIPS: 5375.99
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni vnmi umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_clear serialize ibt flush_l1d arch_capabilities
Virtualization features:
Virtualization: VT-x
Hypervisor vendor: Microsoft

saicharan_reddi@SAICHARAN
Virtualization type: full
Caches (sum of all):
L1d: 288 KiB (6 instances)
L1i: 192 KiB (6 instances)
L2: 7.5 MiB (6 instances)
L3: 12 MiB (1 instance)
NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0-11
Vulnerabilities:
Gather data sampling: Not affected
Ghostwrite: Not affected
Indirect target selection: Not affected
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Old microcode: Not affected
Reg file data sampling: Mitigation; Clear Register File
Retbleed: Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSB-eIBRS SW sequence; BHI BHI_DIS_S
Srbds: Not affected
Tsa: Not affected
Tsx async abort: Not affected
Vmscape: Not affected



Relationship Between Hardware and Operating System Services

1. CPU Abstraction

- The operating system manages CPU execution using the scheduler.
- Each program receives CPU time through time-sharing.
- Users do not directly access the CPU.

Example:
The top command shows CPU utilization and running processes.

---

2. Memory Abstraction

- **The operating system provides virtual memory.**
- **Each process gets its own memory space.**
- **Memory allocation and protection are handled automatically.**

Example:
The top command displays RAM and swap memory usage.

---

3. Storage Abstraction

- **The operating system manages storage through a file system.**
- **Files are stored on disks without requiring users to know physical disk locations.**
- **File operations like create, read, write, and delete are managed by the OS.**

---

4. I/O Device Abstraction

- **The operating system controls communication with devices such as keyboard, mouse, printer, and disk.**
- **Device drivers provide a standard interface for hardware.**
- **Applications use system calls instead of communicating directly with hardware.**

Role of the Operating System

- **Manages CPU scheduling.**
- **Allocates memory to processes.**
- **Controls storage devices.**
- **Handles input/output operations.**
- **Manages running processes.**
- **Provides security and resource protection.**

Conclusion

The Linux operating system acts as an interface between hardware and applications. Using the commands uname, lscpu, ps, and top, we observed how the operating system manages hardware resources such as the CPU, memory, storage, and I/O devices. The OS abstracts these resources, allowing users and applications to use the computer efficiently without directly interacting with the hardware.